Multiplexing Database files

Purpose:

⇒ To minimize the possibility of losing a control file or a redo log file, multiplexing of database files reduces or eliminates data-loss problem caused by media-failures.

How it is done?

⇒ multiplexing can be done:

1. Automated by using Automatic Storage Management (ASM) instance
2. Can be done manually.

1 Automatic Storage management:

⇒ ASM is a multiplexing solution that automates the layout of data files, control files and redo log files by distributing them across all available disks.

⇒ When new disks are added to the ASM cluster, the database file are automatically redistributed across all disk volumes for optimal performance.

⇒ The multiplexing features of an ASM cluster minimize the possibility of data loss and generally more effective than a manual scheme that places critical files and backups on different physical drives.

2. Manual Multiplexing:

⇒ To provide safeguard for critical database files can be done manually.

How?

⇒ By setting some initialization parameters and providing an additional location for control files, redo log files, and archived redo log files.

1. Control files multiplexing

Purpose: Ensures redundancy and availability of control files in Oracle databases.

Multiplexing:

=> Process of creating multiple copies of control files.

1. Immediate multiplexing:

- Can be performed during database creation.
- Allows up to eight copies of a control file.

2. Delayed multiplexing:

=> Control files can be multiplexed later through manual copying to multiple destinations.

Setting CONTROL_FILES parameter:

Initialization parameters:

- Regardless of when multiplexing occurs, the value of CONTROL_FILES remains the same.

Adding multiplexed locations:

- Edit initialization parameter file to add additional locations.

Altering CONTROL_FILES parameter

Using SPFILE:

- Use the ALTER SYSTEM command to modify the CONTROL_FILES parameter in SPFILE.

ex:

```
ALTER SYSTEM
SET   CONTROL_FILES = '  ---  '
SCOPE = SPFILE ;
```

Restarting Database:

⇒ After modifying control file locations, shut down the database, copy control files to new destinations, and restart the database.

Verification:

⇒ Checking Control file Locations

⇒ Use data dictionary views like V$SPPARAMETER and V$CONTROLFILE to verify control file names and locations.

Query:

```
SELECT   Value
From     V$parameter
Where    name = 'control_files';
```

⇒ This query will return one row for each multiplexed copy of the control file.

⇒ The view V$CONTROLFILE contains one row for each copy of the control file along with its status.

2. Redo log files multiplexing

Introduction:

⇒ Redo log files are like journals for the database, keeping track of changes.

⇒ By default, oracle sets up three redo log files.

Multiplexing redo log files:

⇒ Redo log files are multiplexed by changing a set of redo log files into a redo log file group.

⇒ Each group has identical copies of the redo log files.

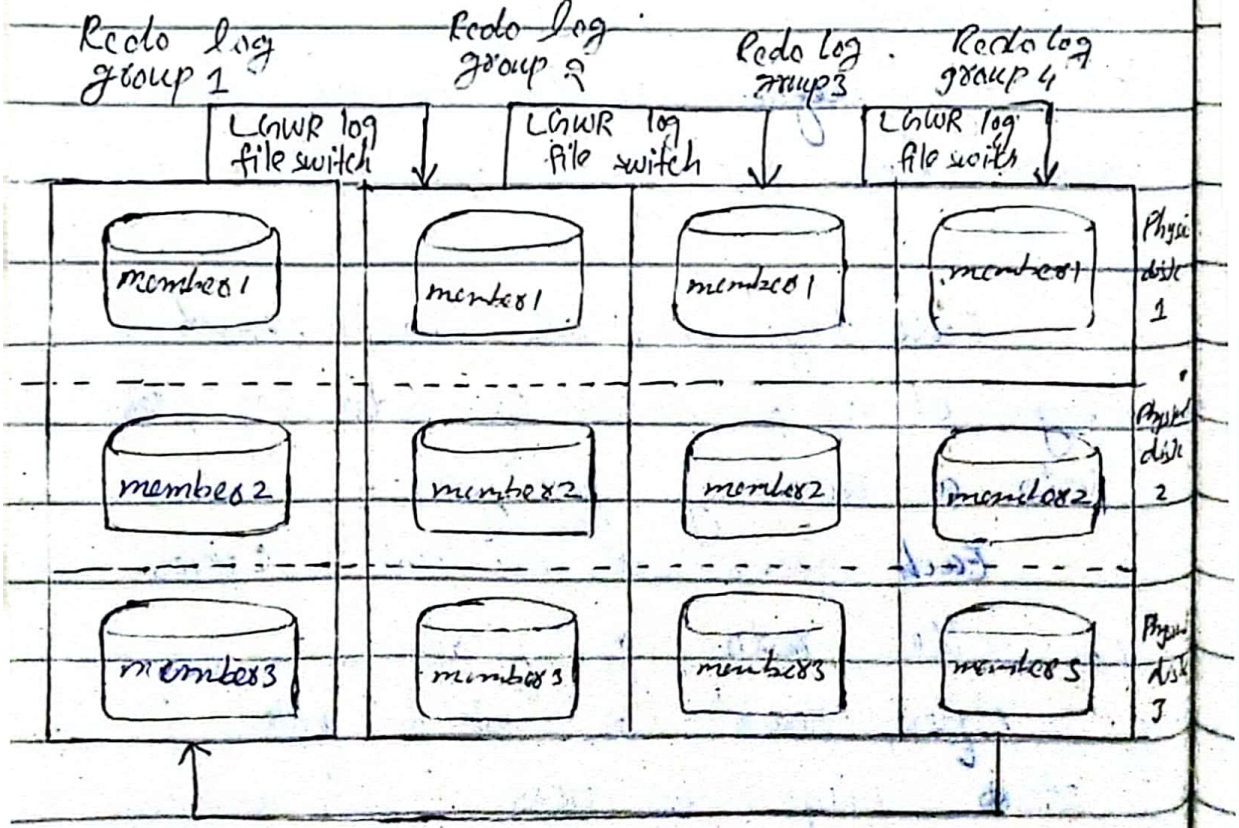
⇒ After each log file is filled, it starts filling the next in sequence.

After the third is filled, the first one is reused.

⇒ To change the set of three redo log files to a group, add one or more identical files as a companion to each of the existing redo log files.

⇒ After the groups are created, the redo log entries are concurrently written to all members within the group.

⇒ When a group is filled, entries overflow to the next group.



=> The following shows how a set of four redo log files can be multiplexed with four groups, each group containing three members.

=> Adding a member to a redo log group is very straightforward.

3. Adding a members to Redolog Groups-

=> Adding a member to a redo log group is simple using alter database command.

=> Specify the name of the new file and.

=> The new file is created with the same size as other members in the group.

Query:

```
alter database
add logfile member '/u05/oracle/dc2/log4y'
to group;
```

4. Expanding Redo log Groups:

=> If redo log files fill up quickly adding another redo log group can be a solution.

⇒ This process involves specifying the group number and creating new log files with the appropriate size.

Syntax:

alter database
add logfile member
'/u05/oracle/dc2/log_3d.dbf'
to group 3;

⇒ alter database
add logfile groups
('/u02/oracle/dc2/log_3a.dbf',
'/u03/oracle/dc2/log_3b.dbf',
'/u04/oracle/dc2/log_3c.dbf') size 250m;

5. Group Configuration:

⇒ All members within a redo log group must be of the same size.

⇒ However, sizes between groups and the number of members within each group can vary.

Example:

⇒ Starting with four redo log groups, adding an extra member to group 3 and creating a fifth group with three members.

6. Redo logfile sizing advisor (oracle 10g):

⇒ Oracle 10g introduced the Redo logfile sizing advisor.

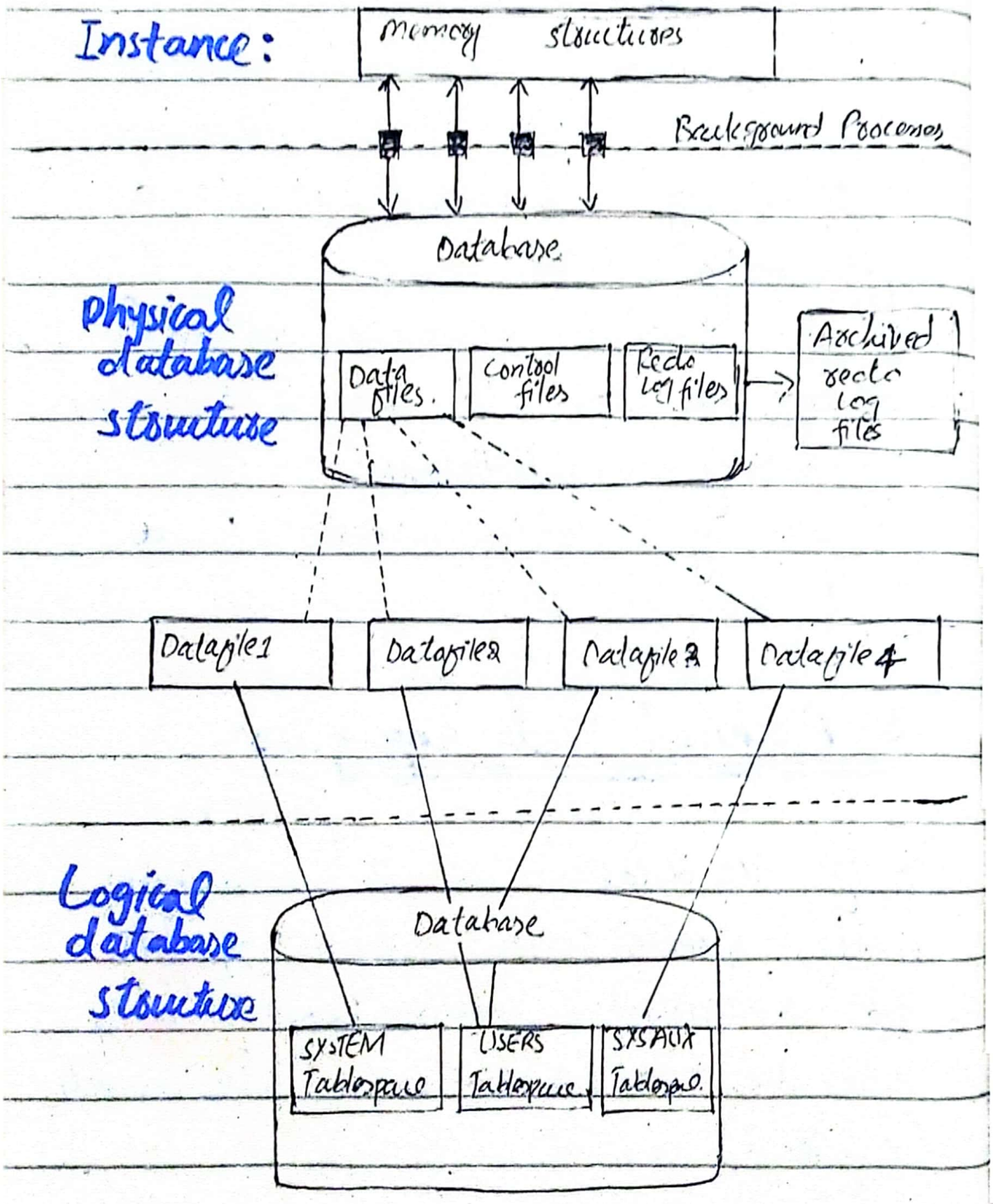
⇒ It helps determine optimal redo log file sizes to prevent excessive I/O activity or bottlenecks.

⇒ Redo logfile sizing advisor is a Tool.

3. Archived Redo Log files:

⇒ if the database is in archiving mode, Oracle copies redo log files to a specified location before they can be reused in the redo log switch cycle.

Q. Draw diagram of oracle Physical storage structures?



Q. Write down Oracle startup and shutdown procedures?

Startup procedure:

⇒ These are the following startup modes:

1. NoMount
2. Mount
3. Open

1. NoMount: (Instance started)

- This is the state when the control file, online redo log files, and the database files are closed and are not accessible.
- The oracle instance is available.

Some of the v$views (dynamic performance views) are available during this state.

⇒ A database may be brought to this state to perform operations:

1. Creating a database.
2. Recreating the control files & manage

Ex: V$session, v$instance, v$database etc.

2. Mount State: (Control file opened for this instance)

- This is the next phase through which the database passes.
- During this stage, the control file is opened and the existence of all the database files and online redo log files is verified.
- This state is typically used for operations like

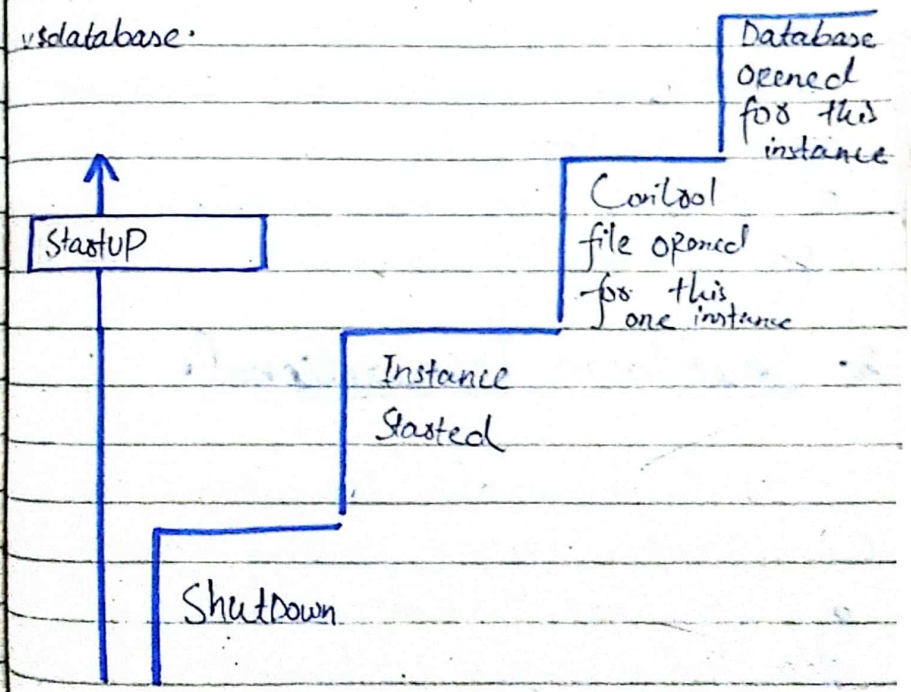
Backup:

- Recovery of the system or undo datafile.
- Change the database to archive mode etc.

⇒ When querying V$database for the open_mode, the response will be "mounted".

3. Open State: (Database opened for this instance)

- The database is opened. During this stage, the data files and online redo log files are opened and ready to use.
- Oracle doesn't allow you to open the database if any of the datafile or online redo log file is missing or corrupted.
- A database may be opened in read-only mode as well as in reading-write mode.
- In read-only mode, DML operations are not permitted but queries are allowed.
- The open mode can be determined by querying the dynamic performance view V$database using the SQL command SELECT open_mode FROM v$database.



Database Shutdown procedures:

⇒ Oracle has three shutdown modes namely:

1. Normal
2. Immediate
3. Abost

1. Shutdown Normal:

- This is the default mode for shutting down the database.
- Oracle waits for all users to disconnect before shutting down.
- Disallows any new connections before shutting down.

2. Shutdown Transactional:

- Waits until all transactions are completed before shutting down.
- No new connections are permitted during this mode.

3. Shutdown Immediate:

- Disconnects all sessions, roll back running transactions, and shuts down the database immediately
- No instance recovery is needed during the next startup.

4. Shutdown Abost:

- This option forcefully shut down the database without rolling back any transactions.
- It's similar to pulling the power plug of a television.
- Subsequent database startup requires instance recovery to be initiated by SMON (System Monitor).

⇒ A clean shutdown sequence involves issuing shutdown abost. followed by startup restrict, shutdown immediate and finally startup mount restrict.

⇒ This ensures a faster shutdown compared to conventional methods, although experienced DBAs typically prefer to wait rather than resort to this approach.

Conclusion:

These procedures are essential for managing Oracle databases effectively, ensuring data integrity and facilitating smooth operations.